Stakeholder Agent System

Design Document

Regis University - MSSE 692: Software Engineering Practicum I

Matt Bass, Anthony Perez, Joe Turner, Ramesh Kondala
2-8-2026

Table of Contents

API Documentation 4
Conventions 4
Content Types 4
Authentication 4
API Versioning 4
Global HTTP Status Codes 4
RESTful APIs 5
AIController 5
PersonaController 5
ProjectController 7
ScenarioController 8
RequirementController 10
Error Handling 11
Standard Error Response Format 11
Custom Exception Mapping 11
Error Response Guidelines 12
Logging 12
Wide Events for Logging 12
Logging Rules: 12
Rate Limiting 13
AI Chat Rate Limiting (POST /api/v1/generate) 13
Data Model & Schema Design 14
Entity-Relationship Diagram 14
Database schema 15
Relationship diagram: 16
Logical Data Model: 16
Physical SQL Schema 17
Indexes 18
Migration strategy 18
Data access patterns 18
Technology Stack Documentation 19
Overview 19

Core Technologies 19
Frontend Platform 19
AI Service Layer 19
Data Layer 19
Supporting Libraries 19
Infrastructure & Deployment 19
Development Environment 19
Component Internal Design 20
Class Diagrams 20
Sequence Diagram 21
Design Patterns 22
Application Architecture Patterns 22
AI Service Architecture Patterns 22
Logging Service Architecture Patterns 22
SOLID Principles 23
Single Responsibility Principle 23
Open/Closed Principle 23
Liskov Substitution Principle 23
Interface Segregation Principle 23
Dependency Inversion Principle 23
Quality Attribute Design Strategies 24
Performance 24
Rate Limiting 24
Horizontal Scaling 24
Security 24
Authentication and Authorization 24
Testability 25
Data Access Layer Testing 25
Integration Testing 25
Real-World Performance Testing 25
Deployability 25
CI/CD Pipeline (Jenkins) 25
Implementation Guidance 26

Development Setup 26
Prerequisites 26
Repository Access 26
Coding Standards 26
Frontend (TypeScript/React) 26
Backend (Python/FastAPI) 27
Testing Approach 27
Frontend Testing 27
Backend Testing 27
Git Workflow 27
Branch Naming 27
Pull Request Process 27
Main Branch Protection 28
Appendix 28
LLM 28

API Documentation

Conventions

Content Types

- Request/response format: JSON
- Content-Type: application/json
- Accept: application/json

Authentication

All endpoints require authentication:

```
Authorization: Bearer <JWT>
```

API Versioning

- *Base path: /api/v1*
- *Versioning: URI versioning*

Global HTTP Status Codes

The following status codes apply across all endpoints unless otherwise specified:

- 200 OK – successful request
- 201 Created – resource successfully created
- 204 No Content – successful deletion
- 400 Bad Request – missing/invalid parameters
- 401 Unauthorized – missing/invalid auth token
- 403 Forbidden – resource exists but not accessible to user
- 404 Not Found – resource not found
- 408 Request Timeout – operation timeout
- 409 Conflict – duplicate resource (e.g., name already exists)
- 429 Too Many Requests – rate limit exceeded
- 500 Internal Server Error – unexpected failure
- 503 Service Unavailable – service temporarily unavailable

RESTful APIs

AIController

```
POST /api/v1/generate
```

Request Schema:

```
{
  "conversation_id": "string",
  "content": "string"
}
```

Response Schema:

```
{
  "conversation_id": "string",
  "content": "string",
  "type": "MessageType",
  "created_at": "timestamp",
  "updated_at": "timestamp"
}
```

Additional Status Codes:

- 503 Service Unavailable – AI provider unavailable / circuit breaker open

---

PersonaController

**Out of Scope – MVP1*

```
GET /api/v1/personas
```

Response Schema:

```
[
  {
    "persona_id": "uuid",
    "name": "string",
    "description": "string",
    "traits": { "key": "value" },
    "created_at": "timestamp",
    "updated_at": "timestamp"
  }
]
```

```
POST /api/v1/personas
```

Request Schema:

```
{
  "name": "string",
  "description": "string",
  "traits": { "key": "value" }
}
```

Response Schema:

```
{
  "persona_id": "uuid",
  "created_at": "timestamp"
}
```

```
PUT /api/v1/personas
```

Request Schema:

```
{
  "name": "string",
  "description": "string",
  "traits": { "key": "value" }
}
```

Response Schema:

```
{
  "persona_id": "uuid",
  "name": "string",
  "description": "string",
  "traits": { "key": "value" },
  "createdAt": "timestamp"
}
```

```
DELETE /api/v1/personas
```

Request Schema:

```
{
  "persona_id": "uuid"
}
```

ProjectController

**Out of Scope – MVP1*

```
GET /api/v1/project
```

Request Schema:

```
{
  "project_id": "string"
}
```

Response Schema:

```
{
  "project_id": "string",
  "name": "string",
  "description": "string",
  "metadata": { "key": "value" },
  "created_at": "timestamp",
  "updated_at": "timestamp"
}
```

```
POST /api/v1/project
```

Request Schema:

```
{
  "name": "string",
  "description": "string",
  "metadata": { "key": "value" }
}
```

Response Schema:

```
{
  "project_id": "string",
  "created_at": "timestamp",
  "updated_at": "timestamp"
}
```

```
PUT /api/v1/project
```

Request Schema:

```
{
  "name": "string",
  "description": "string",
  "metadata": { "key": "value" }
}
```

Response Schema:

```
{
  "project_id": "string",
  "created_at": "timestamp"
}
```

```
DELETE /api/v1/project
```

Request Schema:

```
{
  "project_id": "string"
}
```

ScenarioController

**Out of Scope – MVP1*

```
GET /api/v1/scenario
```

Request Schema:

```
{
  "scenario_id": "string"
}
```

Response Schema:

```
{
  "scenario_id": "string",
  "name": "string",
  "description": "string",
  "initial_context": "string",
  "metadata": { "key": "value" },
  "created_at": "timestamp",
  "updated_at": "timestamp"
}
```

```
POST /api/v1/scenario
```

Request Schema:

```
{
  "name": "string",
  "description": "string",
  "initial_context": "string",
  "metadata": { "key": "value" }
}
```

Response Schema:

```
{
  "scenario_id": "string",
  "created_at": "timestamp",
  "updated_at": "timestamp"
}
```

```
PUT /api/v1/scenario
```

Request Schema:

```
{
  "name": "string",
  "description": "string",
  "initial_context": "string",
  "metadata": { "key": "value" }
}
```

Response Schema:

```
{
  "scenario_id": "string",
  "created_at": "timestamp"
}
```

```
DELETE /api/v1/scenario
```

Request Schema:

```
{
  "scenario_id": "string"
}
```

RequirementController

**Out of Scope – MVP1*

```
GET /api/v1/requirement
```

Request Schema:

```
{
  "requirement_id": "string",
  "conversation_id": "string"
}
```

Response Schema:

```
{
  "requirement_id": "string",
  "conversation_id": "string",
  "type": "functional | non-functional",
  "text": "string",
  "priority": "low | medium | high",
  "metadata": { "key": "value" },
  "created_at": "timestamp",
  "updated_at": "timestamp"
}
```

```
POST /api/v1/requirement
```

Request Schema:

```
{
  "conversation_id": "string",
  "type": "functional | non-functional",
  "text": "string",
  "priority": "low | medium | high",
  "metadata": { "key": "value" }
}
```

Response Schema:

```
{
  "requirement_id": "string",
  "created_at": "timestamp",
  "updated_at": "timestamp"
}
```

```
PUT /api/v1/requirement
```

Request Schema:

```
{
  "conversation_id": "string",
  "type": "functional | non-functional",
  "text": "string",
  "priority": "low | medium | high",
  "metadata": { "key": "value" }
}
```

Response Schema:

```
{
  "requirement_id": "string",
  "created_at": "timestamp"
}
```

```
DELETE /api/v1/requirement
```

Request Schema:

```
{
  "requirement_id": "string"
}
```

Error Handling

Standard Error Response Format

```
{
  "error_code": "ErrorType",
  "message": "string",
  "details": { "key": "value" },
  "correlation_id": "string",
  "timestamp": "timestamp"
}
```

Custom Exception Mapping

- ValidationError → 400 Bad Request + VALIDATION_ERROR
- UnauthorizedError → 401 Unauthorized + AUTH_REQUIRED / AUTH_INVALID
- ForbiddenError → 403 Forbidden + FORBIDDEN_RESOURCE_ACCESS
- NotFoundError → 404 Not Found + CONVERSATION_NOT_FOUND / RESOURCE_NOT_FOUND

- ConflictError → 409 Conflict + CONFLICT
- RateLimitError → 429 Too Many Requests + RATE_LIMITED
- AiTimeoutError → 408 Request Timeout + AI_PROVIDER_TIMEOUT
- AiUnavailableError → 503 Service Unavailable + AI_PROVIDER_UNAVAILABLE
- Unhandled exceptions → 500 Internal Server Error + INTERNAL_ERROR

Error Response Guidelines

- message is safe for end users (no stack traces, no secrets)
- details provide validation feedback (field-level issues)
- Stack traces go to logs only (tagged with correlation_id)

Logging

Wide Events for Logging

Log structured wide events for end-to-end traceability.

Included in Every Log Event:

- timestamp
- level (INFO/WARN/ERROR)
- event_name
- user_id
- endpoint + http_method
- status_code

Event Names:

- api.request.received
- api.request.completed
- auth.token.validated
- auth.token.invalid
- auth.forbidden
- db.query.started / db.query.completed
- ai.request.started
- ai.request.completed
- rate_limit.exceeded
- error.handled

Logging Rules:

- Never log auth tokens, secrets, or credentials
- Avoid logging full message content or full prompts; if needed for debugging, log a redacted or truncated version

---

Rate Limiting

All endpoints require a valid auth token; rate limits are applied per authenticated user.

AI Chat Rate Limiting (POST /api/v1/generate)

Goal: Prevent spamming while keeping the UI responsive. Users shouldn't be able to send messages faster than the pace of human conversation.

Rate Limits:

1. Per conversation throttle: 1 request every 3 seconds per conversation_id (prevents double-click spam and accidental rapid resubmits)
2. Per user cap: 20 requests per minute per user (protects AI provider quotas/cost and prevents abuse across multiple conversations)

Enforcement Behavior:

On violation, return:

- 429 Too Many Requests
- Header: Retry-After: <seconds>

Data Model & Schema Design

Entity-Relationship Diagram

Database schema

```
enum MessageType {
  USER [note: 'User sent message']
  AI [note: 'AI generated message']
}
```

```
Table user {
  id string [primary key]
  name string
  email string
  emailVerified boolean
  image string
  createdAt timestamp
  updatedAt timestamp
}
```

```
Table conversation {
  id string [primary key]
  user_id string [ref: > user.id]
  name string
  createdAt timestamp
  updatedAt timestamp
}
```

```
Table message {
  id string [primary key]
  conversation_id string [ref: > conversation.id]
  user_id string [ref: > user.id]
  content string
  type MessageType
  createdAt timestamp
  updatedAt timestamp

  indexes {
    user_id
    (conversation_id, createdAt)
  }
}
```

Relationship diagram:

Logical Data Model:

User:

Stores application users.

Field	Type	Notes
id	Varchar(PK)	User identifier
email	varchar	unique
name	varchar	optional
Created_at	timestamps	Default now()

Conversation:

It represents a chat thread

Field	Type	Notes
id	Varchar(PK)	Conversation
User_id	Varchar(FK)	References
Scenario_id	Varchar(FK, nullable)	Future use
Persona_id	Varchar(FK, nullable)	Future use
archieved	boolean	Default false
Last_message_at	timestamptz	Updated on new message
Created_at	timestamptz	Default now()

Message

Stores chat messages

Field	Type	Notes
id	Varchar(PK)	Message id
User_id	Varchar(FK)	sender
Conversation_id	Varchar(FK)	Conversation id
Message_type	ennum	USER/AI
role	Varchar	System/tool/user/ai
content	text	Message text
Token_count	int	Cost tracking
Created_at	timestamptz	Default now()

Physical SQL Schema

Search

Welcome | sql

Users > rameshkondala > sql

```sql
-- ENUM
CREATE TYPE message_type AS ENUM ('USER', 'AI');

-- USER
CREATE TABLE app_user (
  id VARCHAR PRIMARY KEY,
  email VARCHAR UNIQUE NOT NULL,
  name VARCHAR,
  created_at TIMESTAMPTZ DEFAULT now()
);


-- SCENARIO
-- CONVERSATION
CREATE TABLE conversation (
  id VARCHAR PRIMARY KEY,
  user_id VARCHAR NOT NULL REFERENCES app_user(id),
  scenario_id VARCHAR REFERENCES scenario(id),
  persona_id VARCHAR REFERENCES persona(id),
  archived BOOLEAN DEFAULT false,
  last_message_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- MESSAGE
CREATE TABLE message (
  id VARCHAR PRIMARY KEY,
  conversation_id VARCHAR NOT NULL REFERENCES conversation(id),
  user_id VARCHAR NOT NULL REFERENCES app_user(id),
  message_type message_type NOT NULL,
  role VARCHAR,
  content TEXT NOT NULL,
  token_count INT,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- INDEXES
CREATE INDEX idx_message_user ON message(user_id);
CREATE INDEX idx_message_conversation_created
  ON message(conversation_id, created_at);

CREATE INDEX idx_conversation_user_last
  ON conversation(user_id, last_message_at DESC);
```

Indexes

Indexes on message by user id, and a composite index of message by conversation id and created date for displaying a conversation's messages chronologically.

A special index be added to the column message.userid to enable easy querying of messages by user hence supporting activities like recent activity audit or debugging.

Moreover, a composite index of message.conversationid and message.createdAt is to be adopted so that it can be possible to quickly retrieve the history of messages of a conversation in chronological order without additional in-memory sorting operations.

Migration strategy

Schema changes will be managed through ORM-based migration tools to ensure version control and deployment consistency across services.

Frontend Application (TypeScript): Schema changes for user, conversation, and message tables will be managed through Drizzle ORM migration scripts. All schema modifications will be captured as versioned migration artifacts, committed to source control, and executed automatically during the CI/CD pipeline.

AI Service (Python): Schema changes for AI service tables will be managed through Alembic. Migration revisions will be generated and synchronized with service deployments to maintain schema consistency across infrastructure layers.

Data access patterns

GetConversations()

Query: `SELECT * FROM conversations WHERE user_id = :user_id`

Purpose: Retrieves all conversations owned by the authenticated user for display in the application sidebar or conversation list.

Dependencies: User schema with conversation ownership relationship

GetMessageHistory(conversation_id)

Query: `SELECT * FROM messages WHERE conversation_id = :conversation_id ORDER BY created_at`

Purpose: Retrieves the complete chronological message history for a specific conversation, utilizing composite indexes for optimized performance.

Dependencies: Conversation schema with message relationships indexed by conversation_id and user_id

Entity Persistence

All create and update operations for user, conversation, and message entities are managed through their respective ORM layers: Drizzle for the frontend/backend application and SQLAlchemy for the AI service. This follows a standardized repository pattern, ensuring consistent data validation and access control across the system.

Technology Stack Documentation

Overview

Our application uses a modern full-stack architecture with TypeScript on the frontend and Python for AI services, both backed by PostgreSQL.

Core Technologies

Frontend Platform

We are building with TypeScript and React using Tanstack Start as our framework, which gives us server-side rendering and routing on top of Node.js. This setup lets us share code between client and server while keeping everything type safe.

AI Service Layer

The AI components run on Python with FastAPI handling the HTTP layer and PydanticAI managing our AI agent logic. We chose FastAPI for its async capabilities and automatic API documentation, while PydanticAI gives us structured, type-safe AI interactions.

Data Layer

PostgreSQL serves as our primary database. On the frontend side, we use Drizzle as our ORM because it's lightweight and TypeScript-native. The AI service uses SQLAlchemy with Alembic handling database migrations and psycopg as the Postgres adapter.

Supporting Libraries

The frontend uses shadcn-ui and Tailwind for the UI layer, which gives us a consistent design system without bloating our bundle. We're using neon-auth-sdk for authentication and dotenv for environment configuration.

On the backend, we manage environment variables with python-dotenv and use Pydantic for data validation throughout the service layer.

Infrastructure & Deployment

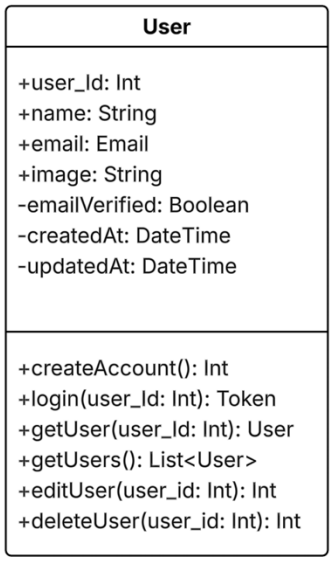
We containerize everything with Docker and deploy to AWS infrastructure. Jenkins handles our CI/CD pipeline, automating builds and deployments.

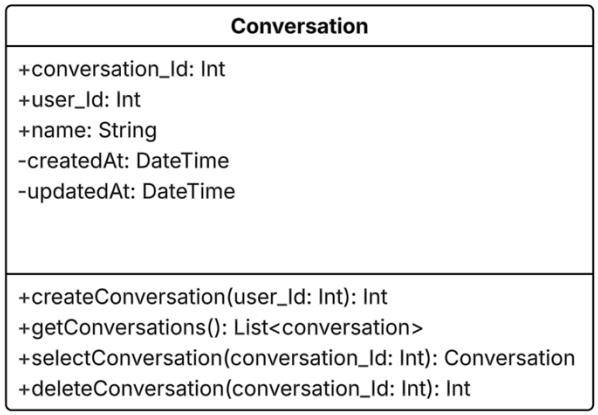
Development Environment

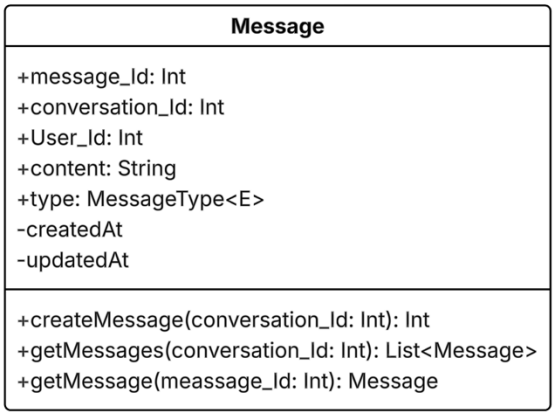
The team uses VS Code for frontend work, PyCharm for Python development, and WebStorm as an alternative for JavaScript/TypeScript. We test API endpoints with Postman or similar tools, and all code lives in GitHub.

Component Internal Design

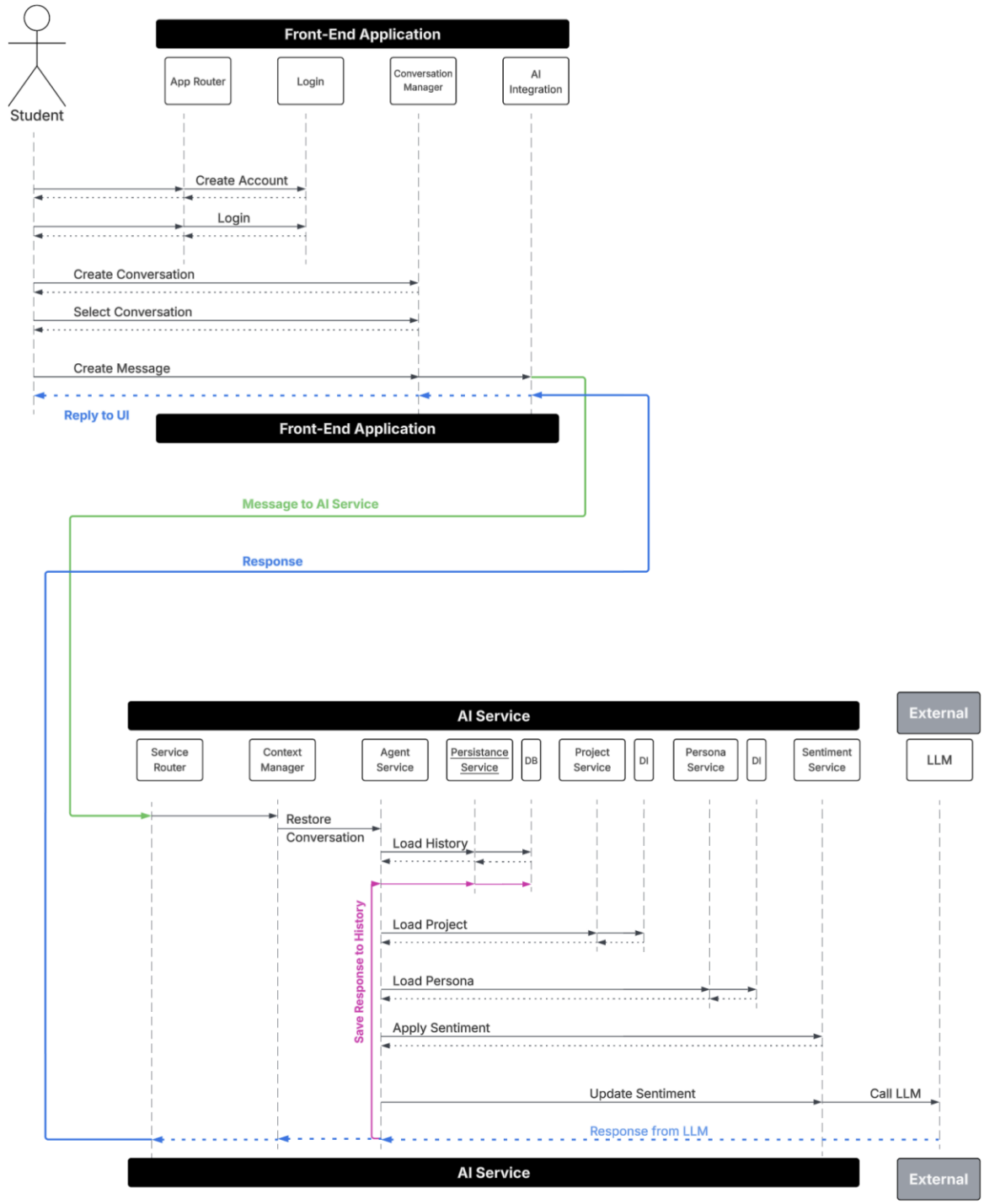
Class Diagrams







Sequence Diagram



Design Patterns

The system architecture and component design feature well-known design patterns from both Martin Fowler's *Patterns of Enterprise Application Development* (2003) and the *"Gang of Four"* commonly referenced patterns:

Application Architecture Patterns

Facade Pattern: Provides a simplified interface for complex subsystem interactions, improving code maintainability and reducing coupling.

Server-Side Rendering (SSR): Enhances initial page load performance and UI responsiveness by rendering content on the server before delivery to the client.

Chain of Responsibility Pattern: Implements sequential processing for registration and authorization workflows, allowing flexible request handling through a chain of handlers.

Client Session State Pattern: Manages stateful functionality on the frontend while keeping backend services stateless, enabling improved UI responsiveness and scalability.

Adapter Pattern: Transforms application-layer calls to conform to service API specifications, ensuring seamless integration between components.

AI Service Architecture Patterns

Gateway Pattern: Encapsulates and centralizes access to external AI services, providing a single-entry point and abstraction layer.

Decorator Pattern: Leverages FastAPI's decorator-based framework for rapid API development and enhanced tooling capabilities.

Builder Pattern: Orchestrates dynamic context reconstruction within the Agent Service, enabling flexible conversation state assembly.

Repository Pattern: Abstracts data persistence operations from business logic, providing a clean separation between ORM interactions and service layer code.

Service Layer: Consolidates and exposes core business capabilities through a well-defined interface, maintaining separation of concerns.

Logging Service Architecture Patterns

Observer Pattern: Implements Wide-Event Logging by allowing multiple subscribers to react to system events without tight coupling.

Mediator Pattern: Centralizes and coordinates conversation-related communications between components, reducing direct dependencies.

SOLID Principles

Single Responsibility Principle

The three core domain classes maintain distinct, well-defined responsibilities:

- User class: Manages user account operations and authentication
- Message class: Handles individual message entities and their properties
- Conversation class: Manages conversation collections and user associations

Open/Closed Principle

Not applicable. The application architecture does not rely on significant inheritance hierarchies.

Liskov Substitution Principle

Not applicable. The application architecture does not rely on significant inheritance hierarchies.

Interface Segregation Principle

API endpoints are designed to reflect specific client requirements, preventing unnecessary dependencies. Additional endpoints have been identified for future development but are explicitly excluded from the MVP scope and will not be implemented within the 8-week development window.

Dependency Inversion Principle

FastAPI's dependency injection system enforces abstraction and contract-based design, ensuring high-level modules depend on abstractions rather than concrete implementations.

Quality Attribute Design Strategies

Performance

Rate Limiting

Rate limits will be enforced to prevent system abuse and ensure fair resource allocation. When limits are exceeded, the system will return an HTTP 429 (Too Many Requests) response with appropriate retry-after headers.

Example implementation: Maximum of 20 messages per minute per authenticated user, with per-conversation throttling of 1 request every 3 seconds.

Design considerations for documentation:

- Rate limit enforcement location (API Gateway vs. application middleware)
- Performance metrics to monitor: 95th percentile latency, requests per second, rate limit hit frequency

Horizontal Scaling

The application is designed for horizontal scalability through stateless API instances deployed behind a load balancer (AWS ECS/Fargate or equivalent). The database layer supports automatic scaling to accommodate traffic fluctuations without manual intervention.

Security

Authentication and Authorization

The system leverages Neon Auth for user management and authentication with the following security model:

Authentication: Neon Auth manages user identity and provides JWT tokens containing user attributes (user_id, email). The application validates these tokens and extracts user context for authorization decisions. Neon provides database-level security and encryption at rest.

Authorization: A role-based access control (RBAC) system governs resource access with the following roles:

- Student: Standard user with access to own conversations and messages
- Instructor: Elevated privileges for managing scenarios and personas (future scope)

Security Documentation Requirements:

- Endpoint-level authorization requirements (which roles or claims are needed per endpoint)
- User identity propagation flow (how user_id is extracted from Neon Auth JWT, passed through application layers, and used in database queries)
- Resource ownership validation (ensuring users can only access their own conversations and messages)

Testability

The application implements a comprehensive testing strategy across three levels: unit testing, integration testing, and real-world performance testing.

Data Access Layer Testing

Test coverage includes:

- Conversation and message creation operations
- Conversation and message retrieval queries
- Unauthorized access attempts and permission validation
- Database constraint enforcement and data integrity

Integration Testing

End-to-end scenarios validate complete user workflows:

- User authentication and session establishment
- Conversation creation and management
- Message transmission and retrieval
- Session resumption and state persistence

Real-World Performance Testing

Load testing simulates production conditions with 30 concurrent users executing GetMessageHistory and SendMessage operations to validate indexing strategies, query performance, and latency requirements under realistic load.

Deployability

CI/CD Pipeline (Jenkins)

The deployment pipeline automates build, test, and deployment processes from GitHub to AWS:

Build Stage: Dependency installation and artifact compilation for both TypeScript application and Python AI service

Test Stage: Execution of unit and integration test suites with immediate pipeline failure on any test failures or database migration errors

Deploy Stage:

- Docker image creation for TypeScript application and Python AI service
- Image publishing to container registry
- Deployment to AWS (ECS/EKS) with database connectivity configuration
- Environment-specific configuration management

Documentation Requirements:

A high-level Jenkins pipeline diagram must be included in the architecture/operations section, illustrating the flow from code commit through deployment. Detailed implementation may follow in subsequent phases.

Implementation Guidance

Development Setup

Prerequisites

- Node.js v20 or higher
- Python 3.11 or higher
- Docker (latest stable)
- PostgreSQL 15+
- Git

Repository Access

- **Frontend:** https://github.com/project-steak-holder/ai-frontend
- **Backend:** https://github.com/project-steak-holder/ai-backend
- **Project Docs:** https://github.com/project-steak-holder/project-docs

Each repository will contain its own README with specific setup instructions once initial scaffolding is complete.

Coding Standards

Frontend (TypeScript/React)

Naming Conventions

- Files: kebab-case (user-profile.tsx)
- Components: PascalCase (UserProfile)
- Functions/Variables: camelCase (getUserData)
- Constants: UPPER_SNAKE_CASE (API_BASE_URL)
- Types/Interfaces: PascalCase (UserProfile)

Code Quality

- Use Biome for linting and formatting
- Follow TypeScript best practices
- Enable strict type checking
- Avoid any types

Backend (Python/FastAPI)

Naming Conventions

- Files: snake_case (user_service.py)
- Classes: PascalCase (UserService)
- Functions/Variables: snake_case (get_user_data)
- Constants: UPPER_SNAKE_CASE (API_VERSION)

Code Quality

- Follow PEP 8 style guide
- Use type hints on all functions
- Document public APIs

Testing Approach

Frontend Testing

- **Framework:** vitest
- **Focus:** Unit tests for business logic, component tests for UI interactions
- Write tests alongside feature development

Backend Testing

- **Framework:** pytest
- **Focus:** Unit tests for services, integration tests for API endpoints
- Write tests alongside feature development

Git Workflow

Branch Naming

- Features: feature/<descriptive-name>
- Bug fixes: bugfix/<issue-description>
- Hotfixes: hotfix/<critical-issue>

Pull Request Process

1. Create Branch

```
git checkout -b feature/<feature-name>
```

2. Submit Pull Request Target the main branch with:

- **Change Description:** What changed and why

- **Build Evidence:** Screenshot showing build and tests passing

3. Review Requirements

- Minimum 1 approval required
- All tests must pass

4. After Approval

- Merge to main
- Delete feature branch

Main Branch Protection

The main branch is protected and requires passing builds plus one approval before merging.

Appendix

LLM

For development of the MVP, an OLlama instance is installed on a team member's reverse-proxied personal server. The Llama LLM was cited in multiple scholarly sources as being particularly well suited for human instruction regarding response parameters and managing large contexts. During quality testing, the performance of multiple LLMs will be compared, and the most appropriate backend model will be leveraged for MVP delivery.

The tentative plan should OLlama prove to be the best performing LLM will likely include hosting the model in the Application's cloud where GPU usage cost accrual will be a major concern. One tentative plan could see the OLlama instance deployed as a headless Kubernetes (EKS, AKS) and auto scale it to zero when idle to manage costs. More research and development will be required should this become needed.